

Django LMS Implementation Guide

Quick Start and Setup Instructions

Prerequisites

Before setting up the Entrepreneurship LMS, ensure you have the following installed:

- Python 3.11 or higher
- Git for version control
- A text editor or IDE (VS Code, PyCharm, etc.)
- Basic familiarity with Django and web development

Installation Steps

1. **Clone the Repository** `bash git clone <repository-url> cd entrepreneurship_lms`
2. **Create Virtual Environment** `bash python -m venv venv source venv/bin/activate # On Windows: venv\Scripts\activate`
3. **Install Dependencies** `bash pip install django djangorestframework pillow django-crispy-forms crispy-bootstrap5 django-allauth`
4. **Configure Database** `bash python manage.py makemigrations python manage.py migrate`
5. **Create Superuser** `bash python manage.py createsuperuser`
6. **Run Development Server** `bash python manage.py runserver 0.0.0.0:8000`

Project Structure

```
entrepreneurship_lms/
├── entrepreneurship_lms/      # Main project directory
│   ├── settings.py           # Django settings
│   ├── urls.py               # Main URL configuration
│   └── wsgi.py               # WSGI configuration
├── accounts/                 # User management app
└── models.py                 # User and profile models
```

```

├── views.py           # Authentication views
├── admin.py          # Admin interface
├── courses/          # Course management app
│   └── models.py     # Course, module, lesson
models
├── views.py          # Course views
├── urls.py           # Course URL patterns
├── admin.py          # Course admin interface
├── content/          # Content management app
│   ├── models.py     # Content and resource models
│   └── admin.py      # Content admin interface
├── assessments/      # Assessment app
│   ├── models.py     # Quiz and assignment models
│   └── admin.py      # Assessment admin interface
├── progress/         # Progress tracking app
│   └── models.py     # Progress and enrollment
models
├── admin.py          # Progress admin interface
├── communication/    # Communication app
│   ├── models.py     # Forum and messaging models
│   └── admin.py      # Communication admin
interface
├── templates/        # HTML templates
│   ├── base.html     # Base template
│   └── courses/       # Course-specific templates
├── static/           # Static files (CSS, JS,
images)
└── media/            # User-uploaded files

```

Key Features Implemented

1. **User Management**
2. Custom user model with extended profiles
3. Role-based access control (Student, Instructor, Admin, Mentor)
4. User skills and goals tracking
5. **Course Management**
6. Hierarchical course structure (Course → Module → Lesson)
7. Multiple content types (text, video, presentations, exercises)
8. Course categories and tagging system
9. **Assessment System**
10. Multiple question types (multiple choice, true/false, essay)
11. Assignment submissions with file uploads

12. Rubric-based grading system

13. **Progress Tracking**

14. Detailed lesson progress monitoring

15. Course completion tracking

16. Achievement and badge system

17. **Communication Tools**

18. Discussion forums

19. Private messaging

20. Announcements and notifications

Admin Interface

Access the Django admin at `/admin/` using your superuser credentials. The admin interface provides:

- User and profile management
- Course creation and editing
- Content management
- Assessment configuration
- Progress monitoring

Next Steps

1. **Add Sample Data:** Create sample courses, users, and content through the admin interface
2. **Customize Templates:** Modify templates in the `templates/` directory to match your branding
3. **Configure Email:** Set up email backend for user notifications and password resets
4. **Add Content:** Upload course materials and create interactive exercises
5. **Test Functionality:** Enroll test users and verify all features work correctly

Development Tips

- Use Django's built-in development server for testing
- Enable debug mode in settings for detailed error information
- Use Django shell (`python manage.py shell`) for testing models and queries
- Check Django documentation for advanced features and best practices

Production Deployment

For production deployment, refer to the comprehensive deployment guide in the main documentation. Key considerations include:

- Use a production-grade database (PostgreSQL/MySQL)
- Configure a web server (Nginx/Apache)
- Set up SSL certificates
- Configure static file serving
- Implement backup procedures
- Set up monitoring and logging